

ENTREGA FINAL

Marcelo Ferreiro Sánchez
Pablo Chantada Saborido
Gillermo García Engelmo
Pepe Romero Conde

28 de noviembre de 2025

Índice

1. Introducción	2
2. Diseño del modelo lógico e implementación del DW en Oracle	2
2.1. Redeseño do modelo da práctica anterior	2
2.2. Toma de decisións	4
2.3. Creación das Táboas	5
2.3.1. Táboas de dimensión	5
2.3.2. Táboas de dimensión SCD	6
2.3.3. Táboas ponte	7
2.3.4. Táboa de feitos	7
3. Diseño e implementación del ETL	8
3.1. Taboas de dimensión	8
3.2. Taboas de dimensión SCD	9
3.3. Taboas ponte	10
3.4. Taboa de feitos	11
3.5. WorkFlow	12
3.6. Variables de Entorno	13
4. Explotación del DW	13
Consulta 1	13
Consulta 2	14
Consulta 3	14

1. Introducción

Este proxecto constitúe a segunda e última fase da práctica sobre deseño e implementación de sistemas de Data Warehouse. Tras completar a primeira entrega, onde se estableceu a arquitectura conceptual e o modelo dimensional inicial, esta segunda parte aborda a construción completa do sistema.

O obxectivo principal é transformar un conxunto de datos reais procedentes de Airbnb nunha infraestrutura analítica funcional e optimizada. Para iso, o traballo estrutúrase en tres partes:

- **Deseño e implementación do modelo lógico:** Realización do modelo en Oracle coas decisións arquitectónicas tomadas para optimizar tanto a complexidade como o rendemento das consultas, incluíndo dimensións, feitos e táboas ponte.
- **Desenvolvemento do proceso ETL:** Automatización completa da extracción, limpeza e carga dos datos dende os ficheiros fonte ata o Data Warehouse mediante Apache Hop, garantindo a calidade e trazabilidade dos datos.
- **Explotación analítica:** Análise do sistema mediante consultas SQL relevantes e visualizacións interactivas en Power BI que permitan extraer coñecementos e Información.

Ao longo desta entrega, documéntase non só o código e as estruturas xeradas, senón tamén as decisións de deseño adoptadas, os problemas resoltos e as funcionalidades avanzadas implementadas. Isto permite non só obter un sistema funcional, senón tamén comprender os fundamentos e as prácticas recomendadas no eido dos Data Warehouses e a intelixencia de negocios.

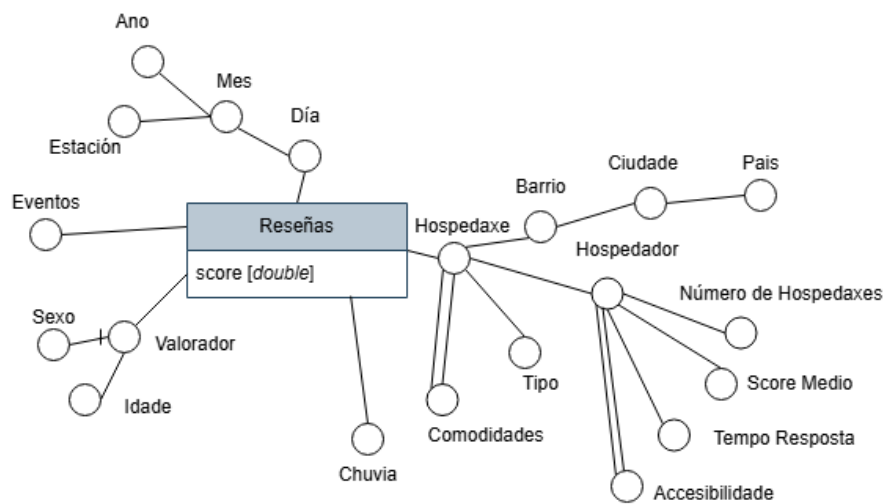
2. Deseño do modelo lógico e implementación del DW en Oracle

2.1. Redeseño do modelo da práctica anterior

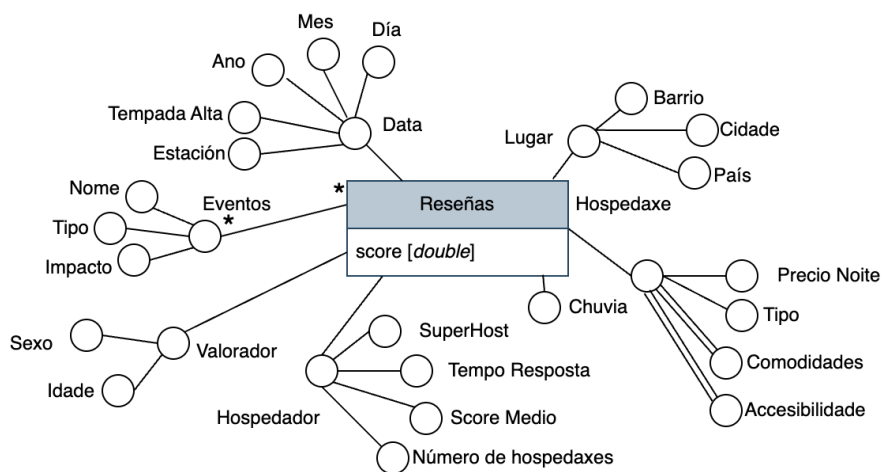
Despois da corrección da entrega anterior modificamos o DFM do seguinte xeito:

- Baixamos as dimensións de **Hospedador** e **Lugar** ó nivel de **Hospedaxe** para evitar o modelo *Copo de neve*, o cal atopamos vantaxoso por gañar simpleza do modelo e velocidade das consultas (ver [1]).
- Actualizamos o DFM cos atributos da dimensión **Eventos** especificados no informe.
- Cambiamos as SCD a tipo 2 para conservar os datos do pasado e poder analízalos (dimensións **Hospedador** e **Valorador**).
- Cambios sobre o resto de atributos para busca coherencia co informe anterior.

Estos cambios pódense ver reflectidos na Figura 1. Na seguinte subsección veremos como implementar estes cambios.



(a) DFM Orixinal, en copo de neve



(b) DFM Modificado, en estrela

Figura 1: Diagrama do Modelo de Feitos Dimensional (*DFM*) antes e despois da corrección

2.2. Toma de decisións

Con todo, resulta un modelo en estrela onde:

- A dimensión **Tempo** está denormalizada, para que en vez de estar representada de forma xerárquica, sea tipo Estrela. Ademáis, como proposto nas teóricas usamos o atributo **data** como chave primaria (identifica o que ten que identificar).
- Á dimensión **Lugar** tivemos que crearlle unha chave primaria *surrogada* pois pode haber, por exemplo, máis dun barrio *chinés*. Con todo, si forzamos a non duplicidade das ternas (**barrio**, **cidade**, **pais**) con **UNIQUE KEY** ¹.
- A dimensión **Hospedaxe** ten dos atributos multievaluados (**Comodidades** e **Accesibilidade**, que describen prestacións e estancias) e modelarase con Táboas Ponte.
- A dimensión **Eventos** usa chaves foráneas para asegurar coherencia e restricións para asegurar que os valores categóricos son os esperados e para evitar duplicados.
- Os atributos **comodidade** e **accesibilidade** ó seren multievaluados teñen que ser modelados cunha táboa ponte. Por tanto cada un merece a súa dimensión propia (ademáis da ponte).
- A dimensión **Chuvia** é dexenerada pois a única métrica que aporta son os mm de precipitación. Polo tanto ben a podemos incluír na táboa de feitos.
- A dimensión **Hospedador** non ten a restricción de non nulidade porque hai hospedadores que levan poucos días e non teñen reseñas.
- A dimensión **Valorador** esixe unha restricción para o sexo: ten que ser Home, Muller, Descoñecido ou Outros. Deste xeito rexistramos a posibilidade de non saber ou ter incertidume sen ter nulos no DW (ver paso de 1a a 1b)
- As SCD, no noso caso Valorador e Hospedador as implementamos a través de engadir as columnas: **actual**, **data_valida_comenzo** **data_valida_fin**. Estas permiten conservar as *filas* anteriores de xeito que sepamos cando estivo vixente cada unha.

¹Non esperamos, en cambio, dous barrios chineses na mesma cidade

2.3. Creación das Táboas

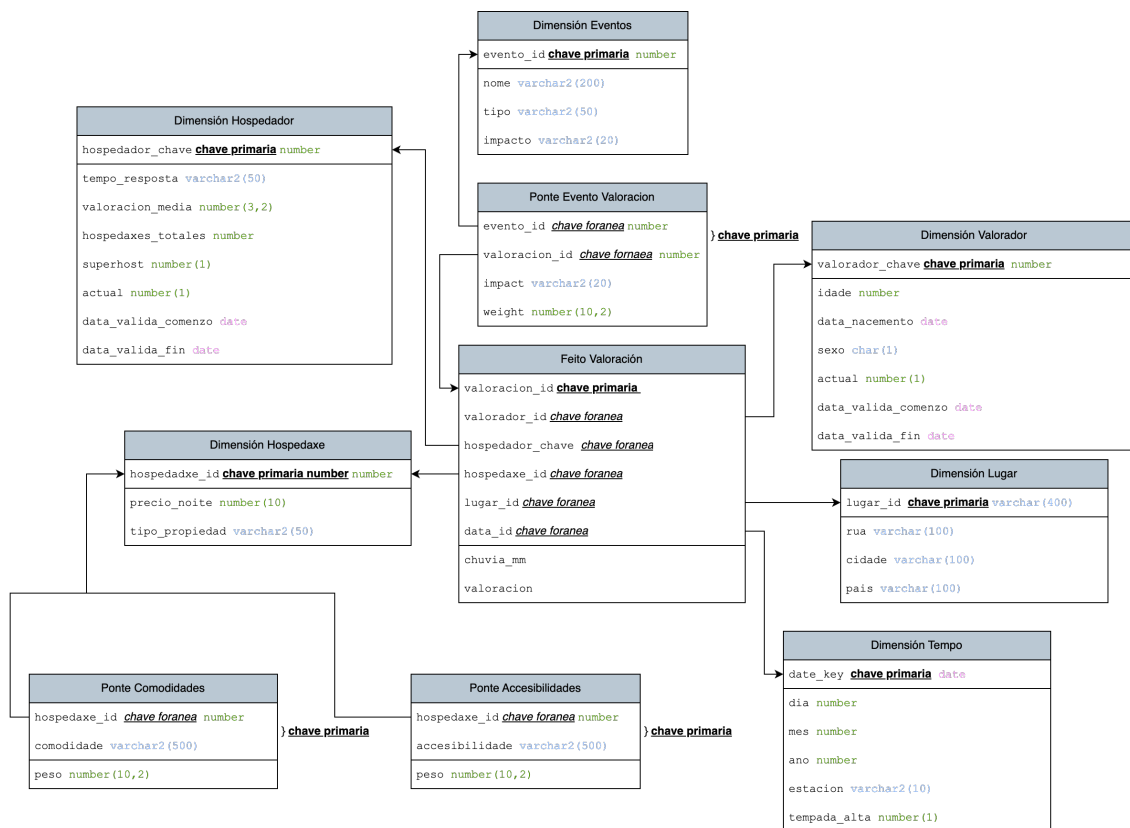


Figura 2: Diseño lóxico

A continuación móstranse as instrucións SQL para crealas táboas. Agrupadas segundo o tipo.

2.3.1. Táboas de dimensión

```
1 CREATE TABLE dim_tempo (
2     data DATE_KEY PRIMARY KEY,
3     dia NUMBER NOT NULL,
4     mes NUMBER NOT NULL,
5     ano NUMBER NOT NULL,
6     estacion VARCHAR2(10) NOT NULL,
7     tempada_alta NUMBER(1) NOT NULL -- 1=TRUE, 0=FALSE);
```

```
1 CREATE TABLE dim_lugar (
2     lugar_id VARCHAR2(100) PRIMARY KEY,
3     barrio VARCHAR2(100) NOT NULL,
4     cidade VARCHAR2(100) NOT NULL,
5     pais VARCHAR2(100) NOT NULL,
6     CONSTRAINT uk_lugar UNIQUE (barrio, cidade, pais));
```

```
1 CREATE TABLE dim_hospedaxe (
2     hospedaxe_id NUMBER PRIMARY KEY, -- Xa a ti amos na OLTP
3     precio_noite NUMBER(10,2) NOT NULL,
```

```

4      tipo VARCHAR2(50) NOT NULL);

1  CREATE TABLE dim_eventos (
2      evento_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
3      ,
4      evento_nome VARCHAR2(200) NOT NULL,
5      evento_tipo VARCHAR2(50) NOT NULL,
6      impacto VARCHAR2(20) NOT NULL,
7      lugar_id NUMBER NOT NULL,
8      evento_data DATE NOT NULL
9
10     CONSTRAINT chk_impacto CHECK (impacto IN ('Bajo', 'Medio'
11     , 'Alto')),
12     CONSTRAINT uk_evento UNIQUE (evento_nome, lugar_id,
13     evento_data)
14
15     CONSTRAINT fk_evento_lugar FOREIGN KEY (lugar_id)
16     REFERENCES dim_lugar(lugar_id),
17     CONSTRAINT fk_evento_tempo FOREIGN KEY (evento_data)
18     REFERENCES dim_tempo(data_key),

```

2.3.2. Táboas de dimensión SCD

```

1  CREATE TABLE dim_valorador (
2      valorador_chave NUMBER GENERATED ALWAYS AS IDENTITY
3      PRIMARY KEY, -- Chave surrogada
4      valorador_id NUMBER NOT NULL, -- ID do OLTP
5      idade NUMBER NOT NULL,
6      sexo CHAR(1) DEFAULT "D" NOT NULL, -- H=Home, M=Muller, D
7      =Desco ecido O=Outro
8      data_nascimento DATE NOT NULL,
9      data_empezou_valer DATE NOT NULL,
10     data_deixou_valer DATE,
11     actual NUMBER(1) DEFAULT 1 NOT NULL, -- 1=TRUE, 0=FALSE
12     CONSTRAINT chk_sexo CHECK (sexo IN ('H','M','D','O')));

```

```

1  CREATE TABLE dim_hospedador (
2      hospedador_chave NUMBER GENERATED ALWAYS AS IDENTITY
3      PRIMARY KEY, -- Chave surrogada
4      hospedador_id NUMBER NOT NULL, -- ID do OLTP
5      tempo_resposta VARCHAR2(50) NOT NULL,
6      score_medio NUMBER(3,2),
7      numero_hospedaxes NUMBER NOT NULL,
8      e_superhost NUMBER(1) DEFAULT 0 NOT NULL, -- 1=TRUE, 0=
9      FALSE
10     data_empezou_valer DATE NOT NULL,
11     data_deixou_valer DATE,
12     actual NUMBER(1) DEFAULT 1 NOT NULL -- 1=TRUE, 0=FALSE);

```

2.3.3. Táboas ponte

```
1 CREATE TABLE ponte_evento_valoracion (
2     evento_id NUMBER NOT NULL,
3     valoracion_id NUMBER NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (evento_id, valoracion_id),
6     FOREIGN KEY (evento_id) REFERENCES dim_eventos(evento_id)
7     ,
8     FOREIGN KEY (valoracion_id) REFERENCES feito_valoracion(
9         valoracion_id));
```

```
1 CREATE TABLE ponte_hospedaxe_commodidade (
2     hospedaxe_id NUMBER NOT NULL,
3     commodidade VARCHAR2(50) NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (hospedaxe_id, commodidade),
6     FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
7         hospedaxe_id));
```

```
1 CREATE TABLE ponte_hospedaxe_accesibilidade (
2     hospedaxe_id NUMBER NOT NULL,
3     accesibilidade VARCHAR2(50) NOT NULL,
4     peso NUMBER(10,2) NOT NULL,
5     PRIMARY KEY (hospedaxe_id, accesibilidade),
6     FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
7         hospedaxe_id));
```

2.3.4. Táboa de feitos

```
1 CREATE TABLE feito_valoracion (
2     valoracion_id NUMBER PRIMARY KEY, -- Do OLTP
3     hospedaxe_id NUMBER NOT NULL,
4     lugar_id NUMBER NOT NULL,
5     data DATE NOT NULL,
6     valorador_chave NUMBER NOT NULL, -- Chave surrogada (SCD
7     Tipo 2)
8     hospedador_chave NUMBER NOT NULL, -- Chave surrogada (SCD
9     Tipo 2)
10    score NUMBER(3,2) NOT NULL,
11    chuva_mm NUMBER(5,2) DEFAULT 0 NOT NULL, -- Dimensi n
12    dexenerada
13    -- Chaves Foraneas
14    FOREIGN KEY (hospedaxe_id) REFERENCES dim_hospedaxe(
15        hospedaxe_id),
16    FOREIGN KEY (lugar_id) REFERENCES dim_lugar(lugar_id),
17    FOREIGN KEY (data) REFERENCES dim_tempo(data),
18    FOREIGN KEY (valorador_chave) REFERENCES dim_valorador(
19        valorador_chave),
```

```
FOREIGN KEY (hospedador_chave) REFERENCES dim_hospedador(
    hospedador_chave));
```

3. Diseño e implementación del ETL

3.1. Taboas de dimensión

Para a dimensión Eventos usamos `events.csv` que xerouse sintéticamente grazas a `generate_events.py`. Nesta taboa xeramos de forma aleatoria co script todos os datos. Os nulos os xestionamos por eliminación de filas.

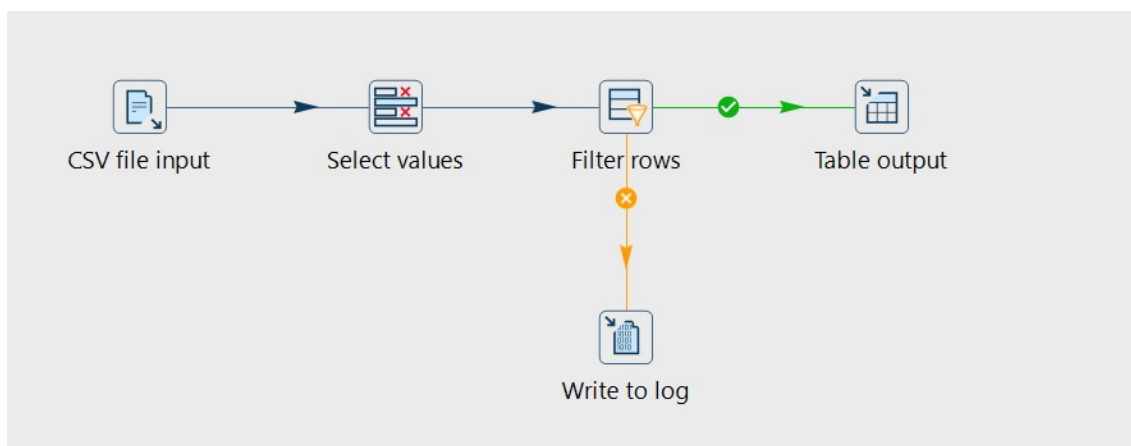


Figura 3: ETL Eventos

Para a dimensión Hospedaxe tivemos que cambiar de formato o `precio_noite`, fixemos unha restricción de non duplicidade. Os nulos os xestionamos por eliminación de filas.

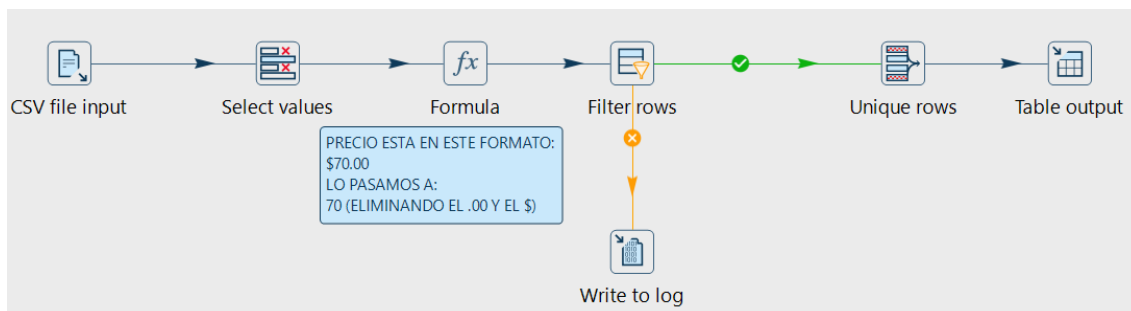


Figura 4: ETL Hospedaxe

A dimensión de lugar tamén esta xerada sintéticamente. Empregando valores de cidades do csv `listings.csv`, xeramos as `STREETS`; os outros dous campos están xerados de forma aleatoria. A chave primaria componse da concatenación dos tres atributos.

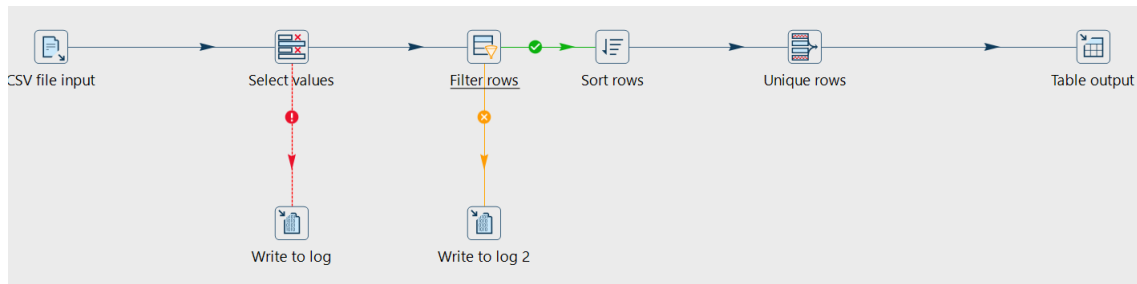


Figura 5: ETL Lugar

Para a dimensión do tempo, creamos filas dende o comezo do dataset ata unos anos máis adelante da última fecha. Desta forma podemos seguir engadindo datos temporales ser ter que modificar a taboa cunha expansión.

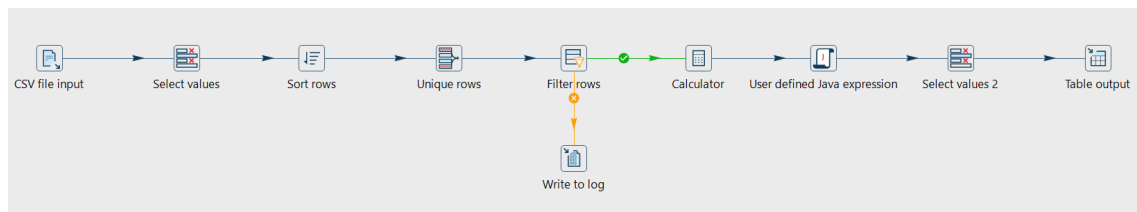


Figura 6: ETL Tempo

3.2. Taboas de dimensión SCD

As dimensións de Hospedador e Valorador forón tratadas de forma externa nos arquivos `generate_host.py` e `generate_reviewers.py`. Para facilitar o precesamento da chaves foráneas e procesos de xeracións de datos para enriquecer as dimensións. Escollimos Python xa que estes procesos complicábanse demasiado no entorno de Apache Hop.

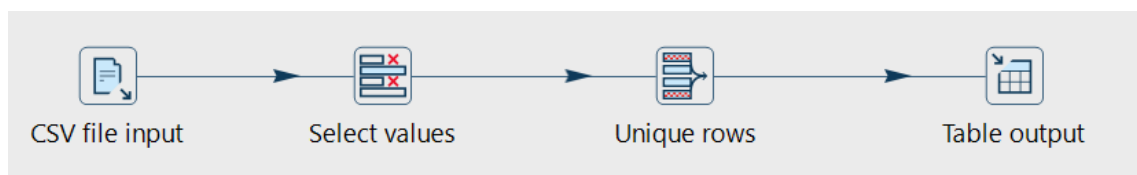


Figura 7: ETL Hospedador

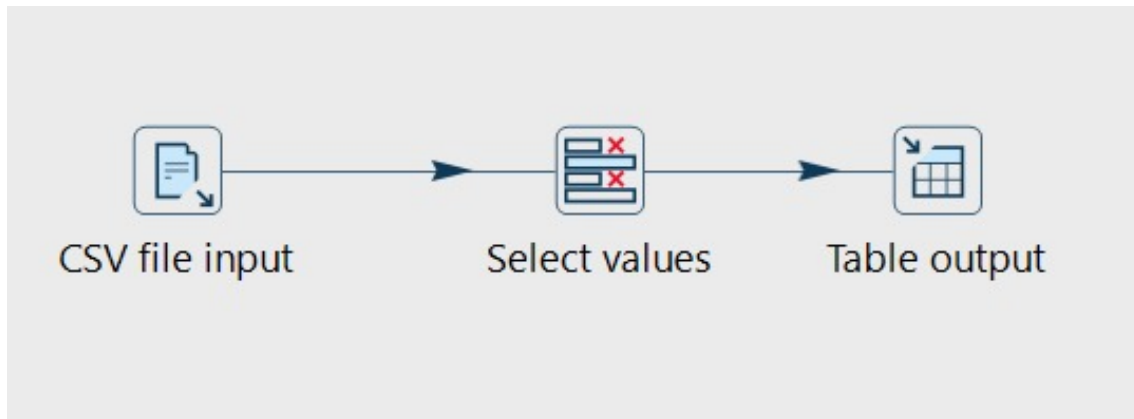


Figura 8: ETL Valorador

3.3. Taboas ponte

Para as taboas ponte empregamos dous aspectos únicos a comparación doutras dimensións. Primeiro aplicamos un atributo de **WEIGHT**, que indica a importancia doutros campos da dimensión segundo o contexto (ao ser principalmente campos sintéticos, estes valores foron postos de forma arbitraria). Ademais, a dimensión de comodidades ten un tratamento especial en Python que agrupa comodidades distintas (32' TV vs. 48' TV) baixo un solo tipo (TV). Por último, obtemos os valores das claves foráneas mediante un Database Lookup, para relacionar os alugares (Hostings) coas súas respectivas accesibilidades dos seus hosts e comodidades que posúen.

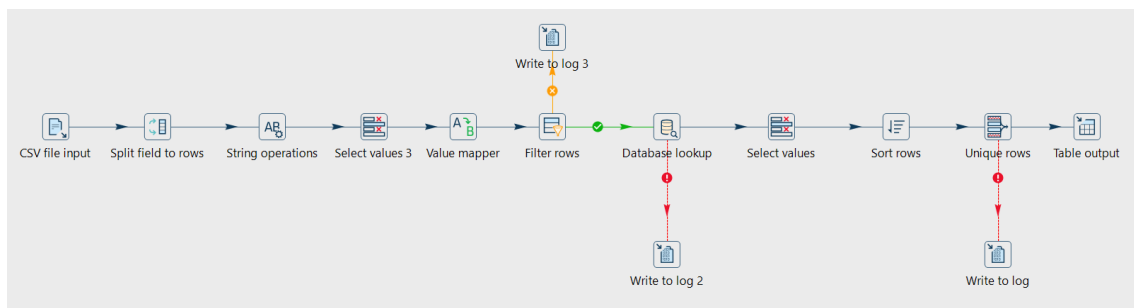


Figura 9: ETL Ponte Accesibilidades

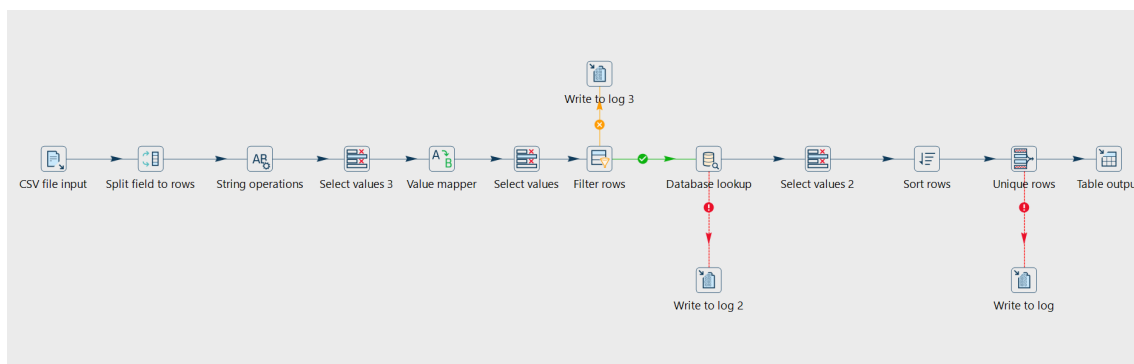


Figura 10: ETL Ponte Comodidades

Na ponte de feito-valoracion realizamos un proceso similar ao das pontes anteriores pero empregando un DATABASE JOIN para obter os identificadores das reviews.

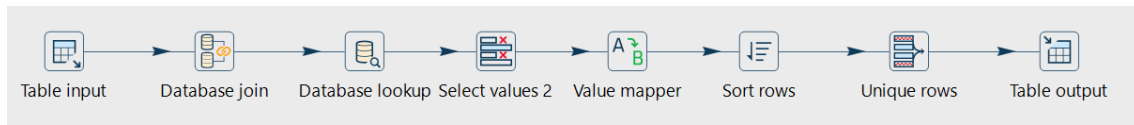


Figura 11: ETL Ponte Evento Valoración

3.4. Taboa de feitos

A taboa de feitos é parcialmente sintetica, tanto a chuva como o score son xerados pornosco. No proceso ETL de Hop obtemos as chaves foráneas mediante DATABASE LOOKUP xa que non precisamos aplicar consultas SQL sobre estes datos no entorno de Apache Hop. Por último, eliminamos as filas con identificadores nulos.

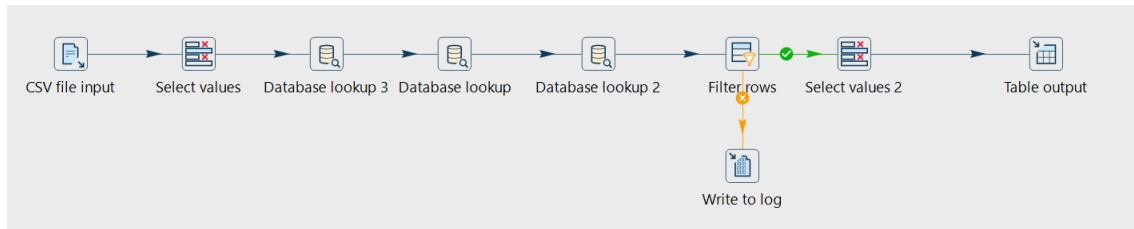


Figura 12: ETL Valoracion

3.5. WorkFlow

No workflow simplemente facemos comprobacións do entorno: comprobación de arquivos, conexión coa base de datos, etc. E a continuación aplicanse os pipelines anteriormente mencionados de forma secuencial. Xerando as taboas correspondientes no proceso.

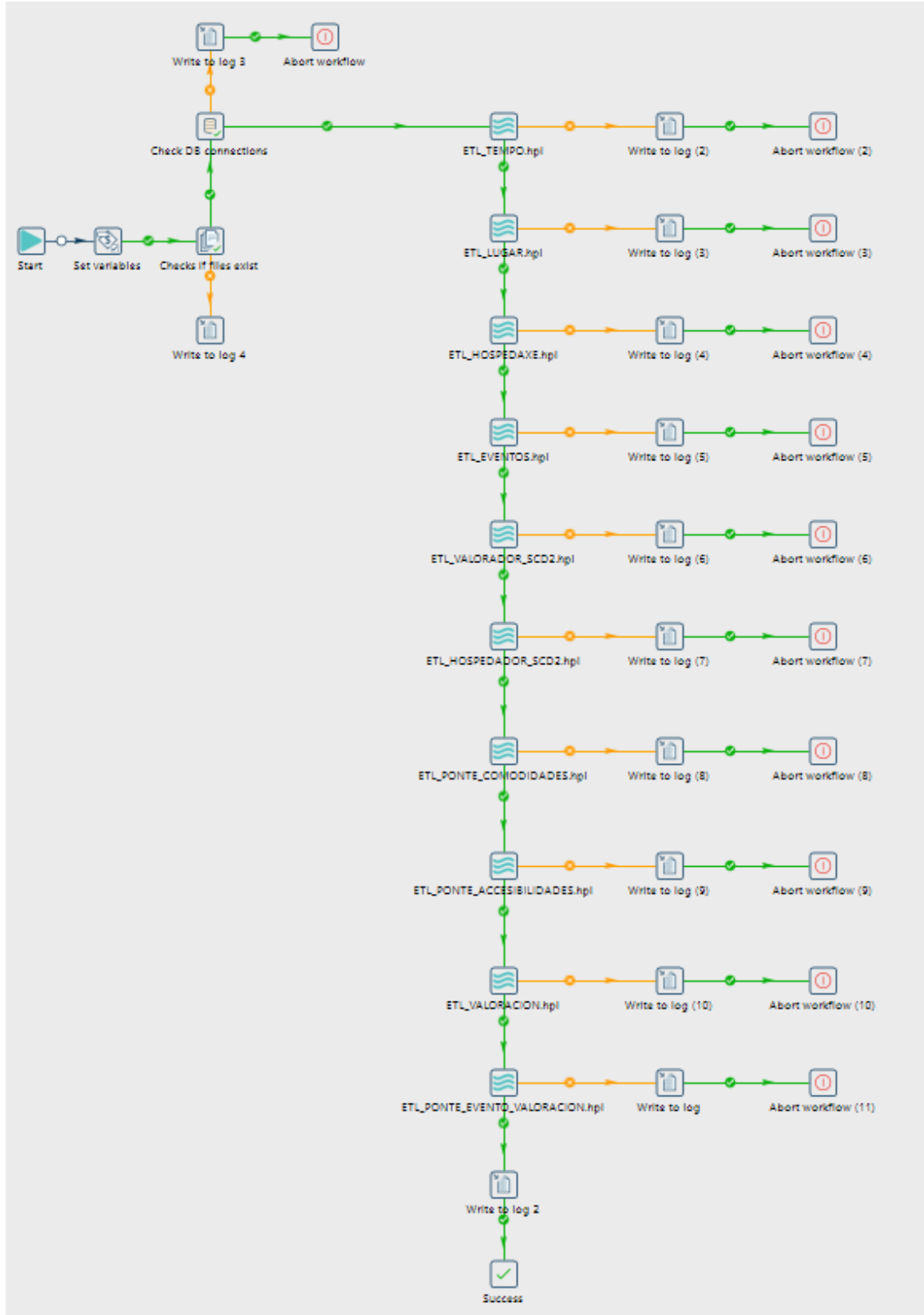


Figura 13: ETL Valoracion

3.6. Variables de Entorno

Para o desarroio do workflow empregamos a seguintes variables de entorno:

- PROJECT_HOME → ABSOLUTE_PATH (ruta da carpeta)
- DATASETS_PATH → PROJECT_HOME / datasets
- ORACLE_USER → 'pablo.chantada'
- ORACLE_PASS → 35634619Y
- ORACLE_HOST → 10.8.48.29

4. Explotación del DW

Consulta 1: Influe o número de hospedaxes dun hospedador sobre o seu score medio? Canto?

Debido á natureza do que significa gráfica é posible creala sen a necesidade dunha consulta SQL. Non será o caso para ó resto de consultas; pero esta resultounos interesante por poder facerse simplemente usando PowerBI. Isto é posible porque ambas métricas son naturais do DW (en cal de derivadas).

Motivación no negocio: Esta consulta é importante para saber caracterizar ós bos e malos hospedadores. Observamos que en xeral a maioría deles teñen poucas propiedades e, dentro dos que teñen moitas, teñen un *score* medio máis baixo. Probablemente sean cadeas contra particulares.

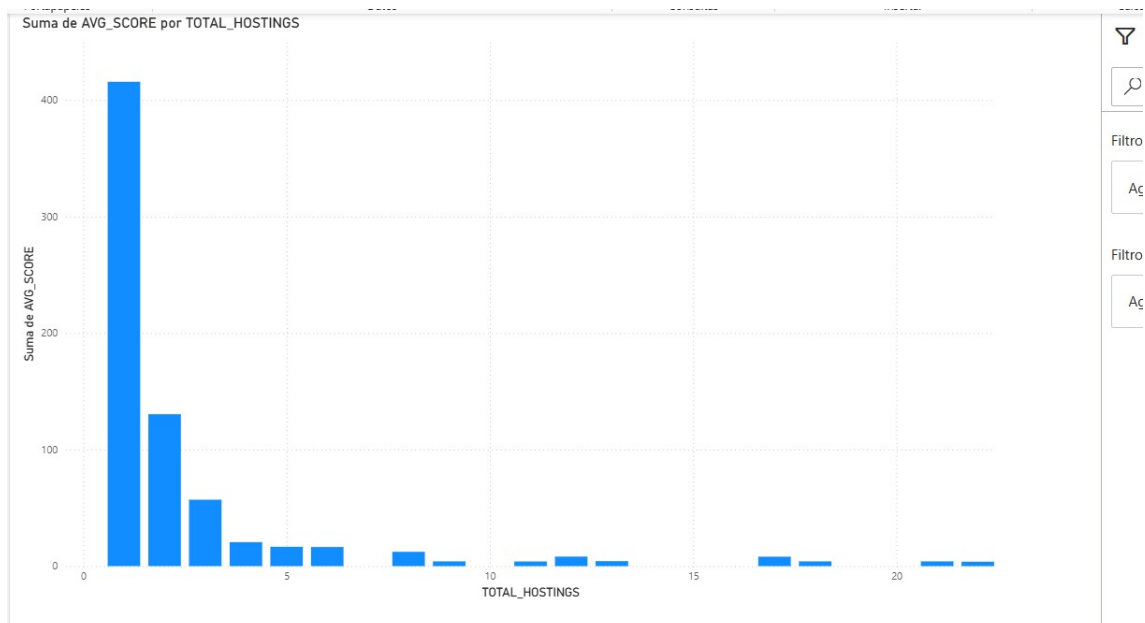


Figura 14: *Score* medio sobre o número de hospedaxes

Consulta 2: Cantas reseñas hai por estación?

Motivación no negocio: Esta consulta pode valer para saber como explotar mellor os recursos e prepararse para as tempadas vindeiras. Na gráfica observamos o que esperaríamos por sentido común: que o verán é máis frecuentado e o inverno é o que menos.

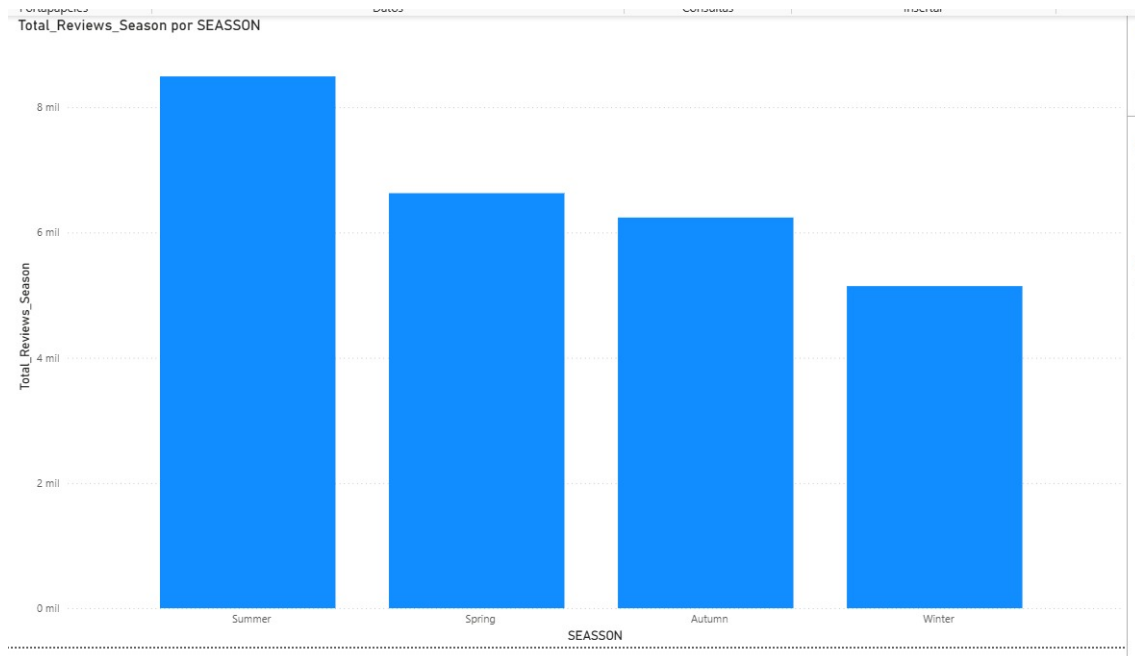


Figura 15: Número total de valoracións por estación

```
1 SELECT
2     t.SEASON,
3     COUNT(DISTINCT v.REVIEW_ID) as Total_Reviews,
4     AVG(h.PRICE_NIGHT) as Avg_Price,
5     SUM(v.SCORE) as Total_Score
6 FROM FEITO_VALORACION v
7 JOIN DIM_TEMPO t ON v.REVIEW_DATE = t.DATE_KEY
8 JOIN DIM_HOSPEDAXE h ON v.HOST_ID = h.HOSTING_ID
9 GROUP BY t.SEASON
```

Consulta 3: Influe a lonxevidade don *hosting* no seu *score* medio?

Motivación no negocio: Podería ser interesante no caso, por exemplo dun host que teña moitos hosts na plataforma (e.g. unha inmobiliaria en cal dun particular) para saber cando podería reformar a propiedade, e volvela anunciar como hosting coa esperanza de aumentar o *score* medio.

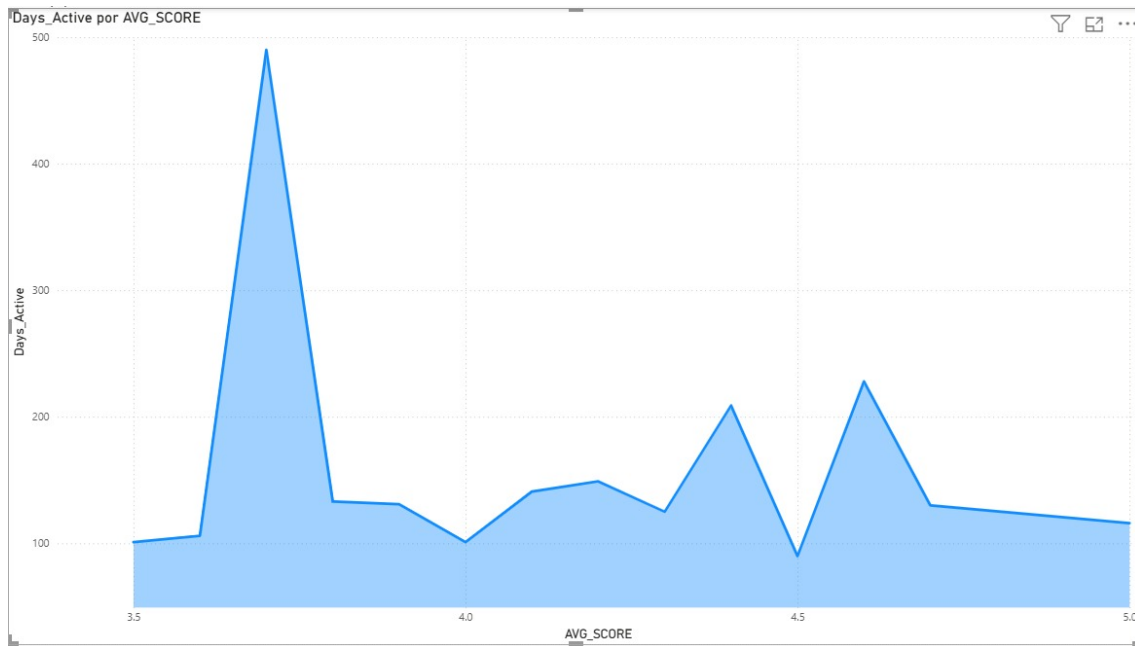


Figura 16: Días activos por *Score* medio.

```
1 SELECT
2     ho.HOST_ID,
3     COUNT(DISTINCT ha.HOSTING_ID) as Total_Properties,
4     AVG(v.SCORE) as Avg_Rating,
5     DATEDIFF(day, ho.DATE_START_VALID, GETDATE()) as
6         Days_Active,
7     ho.SUPERHOST
8 FROM DIM_HOSPEDADOR ho
9 JOIN DIM_HOSPEDAXE ha ON ho.HOST_ID = ha.HOSTING_ID
10 JOIN FEITO_VALORACION v ON ha.HOSTING_ID = v.HOSTING_ID
11 GROUP BY ho.HOST_ID, ho.DATE_START_VALID, ho.SUPERHOST
12 HAVING AVG(v.SCORE) > 4.5
13 ORDER BY Avg_Rating DESC
```

Referencias

- [1] R. Kimball, “Snowflaked dimensions,” 2007, disponible en: <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/snowflake-dimension/>.
Accedido: 2025-11-10.